PHASE 2 · SECTION 04

Implementation Details

UniMentor / Software Engineering · Project structure, dependency management with npm, deployment, database schema and design patterns.

4.1 Project Structure

This section explains how the code base is organised, why each piece of code lives where it does, how dependencies are managed with npm (the Node.js equivalent of Maven or Gradle), and how frontend, backend, database, quality checks and deployment are wired together. It implements the Layered Architecture of Section 3.1 and ADRs 1, 2, 3, 4 and 7.

4.1.1 Repository Layout

UniMentor is a single Git repository (*monorepo*) with two independently deployable applications, plus documentation and infrastructure files:

```
UniMentor/
├── .github/workflows/ci.yml   CI: lint, tests, audit and real-database tests on every PR
├── render.yaml                Render Blueprint: API + PostgreSQL (ADR 7)
├── docs/                      Documentation by phase (+ build/ for the PDFs)
├── backend/                   REST API, Node.js + Express (ADR 1)
│   ├── package.json           Manifest: metadata, scripts, dependencies
│   ├── package-lock.json      Exact dependency tree (generated by npm)
│   ├── eslint.config.js       Static analysis rules
│   ├── .env.example           Template of the environment variables
│   ├── src/
│   │   ├── server.js          Entry point: listen() and graceful shutdown
│   │   ├── app.js             Express app: middleware chain and routers
│   │   ├── config/            env.js (validated settings), db.js (PostgreSQL pool)
│   │   ├── routes/            URL -> controller mapping (index.js mounts every router)
│   │   ├── controllers/       HTTP layer (req/res)
│   │   ├── services/          Business rules
│   │   ├── repositories/      SQL and data access
│   │   ├── middlewares/       Cross-cutting: security chain, errorHandler.js, notFound.js
│   │   ├── utils/             AppError.js, validators.js, sessionCookie.js
│   │   └── database/          SQL migrations, seed data, db:migrate and db:create-admin
│   └── tests/                 Mocked suites, db/ (real PostgreSQL), load/ (Artillery)
└── frontend/                  Static site, vanilla HTML/CSS/JS (ADR 3)
    ├── *.html                 One page per screen (login, tutors, profile, admin...)
    ├── vercel.json            Security headers of the pages
    ├── css/                   base.css (design tokens, shared components), page styles
    └── js/                    config.js, api.js (only module calling fetch), page scripts
```

The applications are separated at the top level because they are built, tested and deployed independently: Render only sees `backend/` and Vercel only serves `frontend/`. They share no code, only a contract: the JSON format of the REST API.

4.1.2 Backend Modularity: Why Each File Lives Where It Does

Each layer of Section 3.1 is a folder with a single responsibility. The guiding rule is: **a module is placed in the layer whose knowledge it needs, and no more.**

Folder	Layer	Knows about	Must NOT know about	Example
`routes/`	API / Routing	URLs, HTTP verbs, route middlewares	Business rules, SQL	`appointmentRoutes.js`
`controllers/`	API / Routing	`req`, `res`, status codes, JSON	SQL, table names	`appointmentController.js`
`services/`	Business Logic	BR-CONF-01..05, use case flow	HTTP, SQL text	`bookingService.js`
`repositories/`	Data Access	SQL, tables, transactions, locks	HTTP, business decisions	`appointmentRepository.js`
`middlewares/`	Cross-cutting	The request pipeline	Specific use cases	`errorHandler.js`, `authenticate.js`
`config/`, `utils/`	Infrastructure	Environment, DB pool, shared helpers	Domain logic	`db.js`, `AppError.js`

Dependency rule. Calls only go *downwards*: `routes → controllers → services → repositories → config/db`. A controller never runs SQL, a service never touches `req`/`res`, and only repositories run SQL (a service may call `db.withTransaction()` to open a transaction that spans several repositories, Section 4.3.2). As a result, a business rule such as BR-CONF-01 (2-hour advance notice) exists in exactly one place and can be unit-tested without HTTP or a database; a schema change (Section 4.2) only affects repositories; and replacing the frontend (ADR 3) would not change the backend.

Request walkthrough (CU-08). The booking sequence diagram (Section 3.2.2-A) maps onto files as follows:

1. `routes/appointmentRoutes.js` matches `POST /api/appointments` and runs the authentication middleware.
2. `controllers/appointmentController.js` reads `{ slotId, sessionType, inquiry }` and the student ID from the JWT, and calls `bookingService.createAppointment(...)`.
3. `services/bookingService.js` applies BR-CONF-01, 04 and 05; if a rule fails it throws `new AppError(409, 'This time slot has just been reserved by another student.')`.
4. `repositories/appointmentRepository.js` runs `BEGIN`, `SELECT ... FOR UPDATE`, `UPDATE availability_slots`, `INSERT INTO appointments`, `COMMIT` inside `db.withTransaction()`.
5. The controller answers `201 Created`; on any error the transaction rolls back and the error middleware sends the JSON error.

The skeleton already contains one complete vertical slice, the health check (`healthRoutes.js → healthController.js → healthService.js → healthRepository.js`), which every team member uses as the template for new use cases.

Why layers and not feature folders. The alternative is one folder per feature (`booking/`, `auth/`) containing all layers. We chose layer folders because (a) Section 3.1 and NFR-04 define the architecture in terms of layers, so the tree makes it visible and checkable; (b) with 14 use cases each layer folder stays small; and (c) the design patterns of Section 4.3 are layer-wide (Repository in `repositories/`, Chain of Responsibility in `middlewares/`). Inside each layer, files are named after their feature, so finding the code of one use case stays easy.

`app.js` vs `server.js`. `app.js` configures Express but does not open a port; `server.js` calls `listen()` and handles `SIGTERM` (sent by Render on every deploy) by closing the server and the database pool. The separation lets tests send requests to the app in memory with Supertest.

4.1.3 Dependency Management with npm

Maven and Gradle declare libraries, download them from a central repository and fix their versions. In Node.js this is done by **npm** and the npm registry, through two committed files: `package.json` and `package-lock.json`. The downloaded code (`node_modules/`) is never committed (`.gitignore`); it is rebuilt from those files.

`package.json`, the manifest. Written by the team, it declares metadata, the required runtime (`"engines": { "node": ">=22" }`), `"private": true` (prevents accidental publishing) and the **scripts**, which play the role of Maven goals or Gradle tasks. Developers, CI and Render run the same commands:

```
"scripts": {
  "start": "node src/server.js",      "dev": "nodemon src/server.js",
  "lint": "eslint .",                 "test": "jest",
  "test:coverage": "jest --coverage", "db:migrate": "node src/database/migrate.js"
}
```

`dependencies` vs `devDependencies`. The criterion is: *does the running application need the package in production?*

Package	Type	Purpose
`express`	dependency	HTTP framework (ADR 1)
`pg`	dependency	PostgreSQL driver, parameterized queries (ADR 2, NFR-02)
`cors`, `cookie-parser`	dependency	Cross-site requests with the HttpOnly JWT cookie (ADR 4)
`jsonwebtoken`	dependency	Signing and verifying JWTs (CU-02)
`bcryptjs`	dependency	Password hashing with cost 12 (CU-01, CU-02, NFR-02); pure JavaScript, so nothing is compiled on Windows or Render

Package	Type	Purpose
`helmet`	dependency	Security headers on every API response (NFR-02)
`dotenv`	dependency	Loads `.env` in local development
`eslint`, `@eslint/js`, `globals`	devDependency	Static analysis (NFR-04)
`jest`, `supertest`	devDependency	Unit and HTTP integration tests (NFR-03, NFR-04)
`nodemon`	devDependency	Auto-restart during development

Developers and CI install both groups (CI needs the linter and tests). Render installs with `npm ci --omit=dev`, so test and lint tools are never deployed, reducing size and attack surface.

Semantic Versioning. Versions are `MAJOR.MINOR.PATCH`: MAJOR may break the API, MINOR adds compatible features, PATCH only fixes bugs. The prefix sets the accepted range:

Range	Accepts	Example
`^5.3.0` (caret, npm default)	`>=5.3.0 <6.0.0`	`express`: 5.x fixes and features, never Express 6
`~5.3.0` (tilde)	`>=5.3.0 <5.4.0`	Patch updates only, for very sensitive packages
`5.3.0` (exact)	Only that version	Not needed: the lock file pins versions

For `0.x` versions the caret is stricter (`^0.4.1` means `<0.5.0`), because any `0.x` release may break. We use `^` everywhere: compatible fixes can be adopted deliberately with `npm update`, while the lock file keeps installs reproducible.

`package-lock.json`, reproducibility. A range alone could resolve differently on different days. The lock file, generated by npm and committed, records the exact version of every package in the tree, including transitive ones (our 14 direct dependencies expand to several hundred packages). For each one it stores the `version`, the `resolved` download URL, an `integrity` SHA-512 hash (npm rejects a package whose content does not match, protecting against tampering) and `dev: true` when it is only needed for development. The two install commands use it differently:

- `npm install` (developers): resolves ranges and **updates** the lock file. Used to add packages (`npm install pg`, `npm install -D jest`).
- `npm ci` (CI and Render): installs **exactly** what the lock file says and fails if it disagrees with `package.json`, so the code tested in CI is the code that runs in production.

Dependency security (NFR-02). CI runs `npm audit --omit=dev --audit-level=high` and fails on high or critical vulnerabilities in production packages (currently 0).

4.1.4 Core Integration

- **Configuration (`config/env.js`).** The only module that reads `process.env`. It gives defaults for development and **fails at startup in production** if `DATABASE_URL`, `JWT_SECRET` or `FRONTEND_ORIGIN` is missing. Secrets live in `.env` (ignored by Git) or in the Render dashboard; `.env.example` documents them.
- **Database connection (`config/db.js`, ADR 2).** One shared `pg.Pool`, so connections are reused (NFR-01). It exposes `query(text, params)`, always with `$1, $2...` placeholders (NFR-02), and `withTransaction(work)`, which runs `BEGIN`, the work, then `COMMIT`, or `ROLLBACK` if anything throws, always releasing the client. It is the base for the booking transaction with `SELECT ... FOR UPDATE` (CU-08, G2, NFR-03) and the suspension transaction (CU-13).
- **Error handling.** Services throw `AppError(status, message, code)`. Express 5 forwards rejected promises from `async` handlers automatically, so controllers need no `try/catch`. `notFound.js` turns unknown routes into 404s, and `errorHandler.js` returns one JSON format, `{ "error": { "status", "message", "code" } }`, hiding internal details of 5xx errors in production.
- **Cross-site authentication (ADR 4).** Frontend (Vercel) and API (Render) are on different sites and the JWT travels in an HttpOnly cookie. The skeleton enables `cors({ origin: FRONTEND_ORIGIN, credentials: true })` (other origins are blocked), `cookie-parser` (the auth middleware reads `req.cookies`) and `trust proxy` (Render terminates HTTPS in a proxy, so this makes Express see the real protocol and client address).
- **Security headers (NFR-02).** `helmet` adds HSTS, `X-Content-Type-Options: nosniff`, `X-Frame-Options` and a Content-Security-Policy of `default-src 'none'` to every API response, since the API only returns JSON. The pages get their own policy from Vercel (`frontend/vercel.json`): scripts and styles only from the site itself (plus `cdn.jsdelivr.net` for FullCalendar, ADR 5), `fetch` only to the site and `*.onrender.com`, no inline scripts and no framing. Together with writing user text through `textContent`, it is the XSS mitigation required by NFR-02.
- **Frontend client (`frontend/js/api.js`).** No page calls `fetch()` directly. The module adds `credentials: 'include'` so the browser sends the cookie while JavaScript never sees the token; serialises JSON; adds the `X-Requested-With` header, which cannot be sent cross-site without passing CORS and supports the CSRF protection of ADR 4; turns error responses into an `ApiError`; and redirects to the login page on `401` (CU-11, A1). Usage: `await api.get('/tutors/' + id + '/availability')`. `config.js` selects the API URL (local or Render) without a build step (ADR 3).
- **Base CSS (`frontend/css/base.css`).** Design tokens as CSS variables (colours, typography, spacing), a reset and shared components (cards, buttons, form fields, badges, toasts), including the slot colours for FullCalendar (ADR 5): available slots in green (CU-04, CU-07).

4.1.5 Quality Assurance, Deployment and Workflow

- **ESLint** (`eslint.config.js`, flat configuration required since ESLint 9): recommended rules plus `prefer-const`, `no-var` and `eqeqeq`.
- **Jest + Supertest**: two suites. `npm test` mocks the database and covers rules, routes and security without PostgreSQL. Coverage threshold: 70 % of lines (NFR-04); current coverage is about 98 %. `npm`

`run test:db` runs the real app against a real PostgreSQL, which a mock cannot replace: migrations, constraints, SQL queries and concurrent requests.

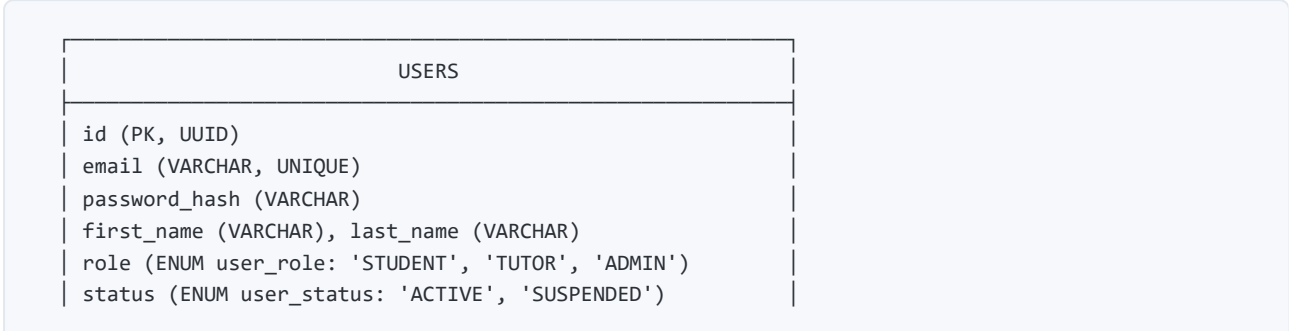
- **Load test (NFR-01).** `tests/load/directory.yml` runs Artillery with 50 concurrent virtual users and fails if the 95th percentile exceeds 200 ms. A local run with 300 tutors measured 79 ms; the deployed service has to be measured as well, because Render's free plan is slower.
- **GitHub Actions** (`ci.yml`): on every push to `main` and every Pull Request it runs two jobs: `npm ci → lint → test:coverage → npm audit`, and the real-database suite against a PostgreSQL 17 service.
- **Deployment (ADR 7).** `render.yaml` defines the `unimentor-api` web service (`rootDir: backend`, `npm ci --omit=dev`, health check `/api/health`) and the `unimentor-db` PostgreSQL database; `DATABASE_URL` is injected from the database and `JWT_SECRET` is generated by Render. The start command is `npm run db:migrate && npm start`, so pending migrations are applied first, and a failed migration prevents the new version from starting. The first administrator is created with `npm run db:create-admin`, so no production password is stored in the repository. Vercel serves `frontend/` as static files over its CDN. Both redeploy on every push to `main`. The landing page calls `/api/health` and `/api/health/db` through `api.js`, showing whether the whole chain (browser → Vercel → Render API → PostgreSQL) works; `/api/health/db` answers 503 if the database is reachable but its tables do not exist.
- **Workflow.** The team's rule is one branch and one Pull Request per use case (`feature/cu-08-booking`), reviewed by another member and merged only with a green CI. A new use case adds its four files (route, controller, service, repository), mounts its router in `routes/index.js` and adds tests, following the health module.

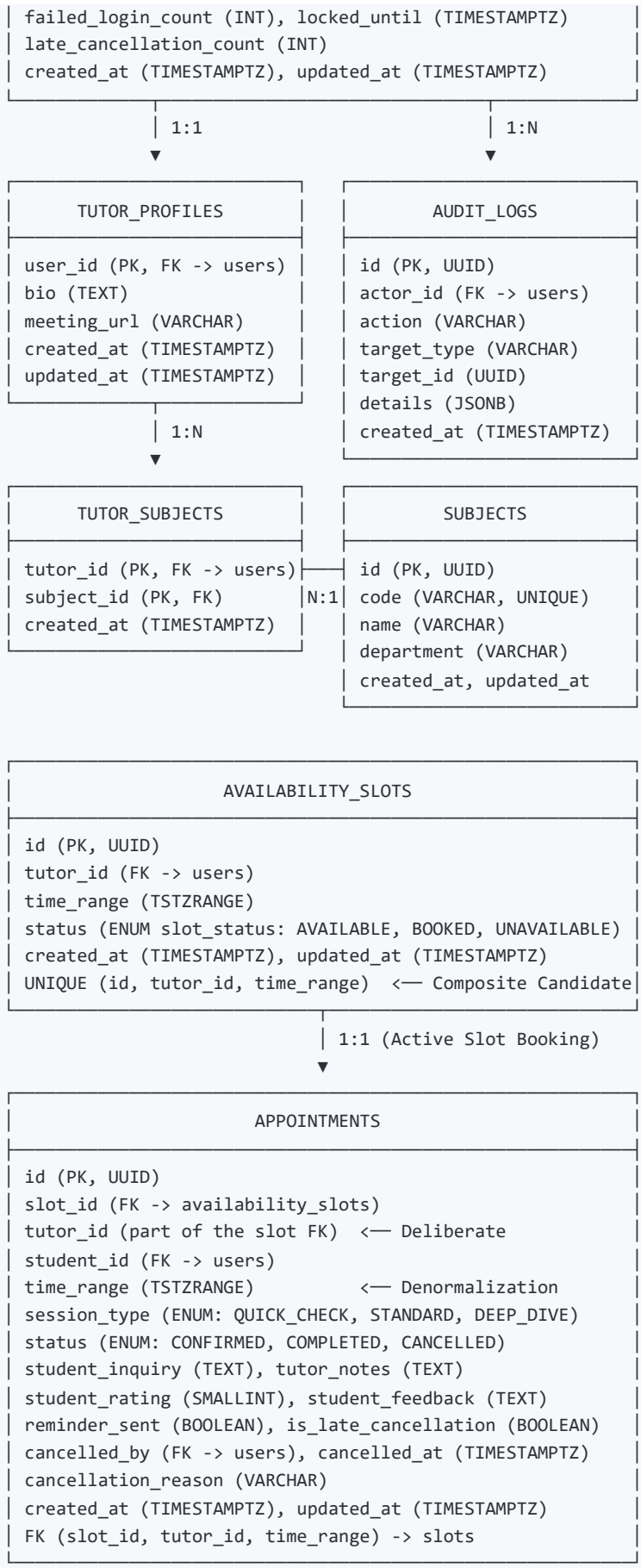
4.2 Database Schema and Persistence Architecture

The persistence layer of the UniMentor platform is implemented on PostgreSQL 16/17. It enforces data integrity, business rules, and concurrency guarantees directly within the database engine rather than relying solely on application-level checks. This implementation realizes ADR 2 (PostgreSQL), which was chosen precisely for its transactional guarantees and engine-level constraints.

4.2.1 Relational Data Model & Entity-Relationship Structure

The relational model consists of seven primary tables structured to satisfy referential integrity, authentication constraints, scheduling rules, and audit requirements:




failed_login_count (INT), locked_until (TIMESTAMPTZ)
late_cancellation_count (INT)
created_at (TIMESTAMPTZ), updated_at (TIMESTAMPTZ)
1:1
1:N
TUTOR_PROFILES
user_id (PK, FK -> users)
bio (TEXT)
meeting_url (VARCHAR)
created_at (TIMESTAMPTZ)
updated_at (TIMESTAMPTZ)
AUDIT_LOGS
id (PK, UUID)
actor_id (FK -> users)
action (VARCHAR)
target_type (VARCHAR)
target_id (UUID)
details (JSONB)
created_at (TIMESTAMPTZ)
1:N
TUTOR_SUBJECTS
tutor_id (PK, FK -> users)
subject_id (PK, FK)
created_at (TIMESTAMPTZ)
N:1
SUBJECTS
id (PK, UUID)
code (VARCHAR, UNIQUE)
name (VARCHAR)
department (VARCHAR)
created_at, updated_at
AVAILABILITY_SLOTS
id (PK, UUID)
tutor_id (FK -> users)
time_range (TSTZRANGE)
status (ENUM slot_status: AVAILABLE, BOOKED, UNAVAILABLE)
created_at (TIMESTAMPTZ), updated_at (TIMESTAMPTZ)
UNIQUE (id, tutor_id, time_range) <— Composite Candidate
1:1 (Active Slot Booking)
APPOINTMENTS
id (PK, UUID)
slot_id (FK -> availability_slots)
tutor_id (part of the slot FK) <— Deliberate
student_id (FK -> users)
time_range (TSTZRANGE) <— Denormalization
session_type (ENUM: QUICK_CHECK, STANDARD, DEEP_DIVE)
status (ENUM: CONFIRMED, COMPLETED, CANCELLED)
student_inquiry (TEXT), tutor_notes (TEXT)
student_rating (SMALLINT), student_feedback (TEXT)
reminder_sent (BOOLEAN), is_late_cancellation (BOOLEAN)
cancelled_by (FK -> users), cancelled_at (TIMESTAMPTZ)
cancellation_reason (VARCHAR)
created_at (TIMESTAMPTZ), updated_at (TIMESTAMPTZ)
FK (slot_id, tutor_id, time_range) -> slots

4.2.2 Normalization Analysis and Deliberate Denormalization

The relational schema has been designed in compliance with classical relational theory and evaluated against Codd's Normal Forms:

1. First Normal Form (1NF)

A relation is in 1NF if every field contains only atomic (indivisible) values and there are no repeating groups.

- **Attribute Atomicity**: Every column across all entities (`email`, `first_name`, `last_name`, `code`, `meeting_url`) holds a single, indivisible scalar value.
- **Absence of Repeating Groups**: Tutor academic specializations are not stored as serialized JSON arrays or delimited strings in `tutor_profiles`. Instead, each qualification is modeled as an individual tuple in the junction table `tutor_subjects`.

2. Second Normal Form (2NF)

A relation is in 2NF if it satisfies 1NF and all non-prime attributes are fully functionally dependent on the entire primary key.

- In `tutor_subjects`, the primary key is composite: `(tutor_id, subject_id)`. The only non-key attribute is `created_at`, which documents the timestamp when that specific association was established. Descriptive subject data (`name`, `department`) is kept exclusively in `subjects`, avoiding partial key dependencies.
- All other tables (`users`, `tutor_profiles`, `subjects`, `availability_slots`, `appointments`, `audit_logs`) use single-column primary keys (`id` or `user_id`), rendering partial functional dependency structurally impossible.

3. Third Normal Form (3NF) and the Intentional Denormalization of `appointments`

A relation is in 3NF if it satisfies 2NF and contains no transitive dependencies (no non-prime attribute depends on another non-prime attribute).

- **Role Specialization**: Tutor-specific metadata (`bio`, `meeting_url`) is segregated into `tutor_profiles` via a 1:1 relation referencing `users(id)`. Placing these attributes directly in `users` would introduce null values for student accounts and violate functional cohesion.
- **Strategic Denormalization in `appointments`**: Under strict 3NF rules, the `appointments` relation would only require `slot_id` and `student_id`. The attributes `tutor_id` and `time_range` are transitively dependent on `slot_id` through `availability_slots`: `slot_id → (tutor_id, time_range)`. However, removing these columns would mean that enforcing business rule **BR-CONF-05** (preventing a student from booking two concurrent appointments across different tutors) could only be achieved via:

1. An application-level read-before-write query, which is vulnerable to race conditions under concurrent client execution.
2. A heavy PL/pgSQL lock trigger escalating table-level locks.

To solve this concurrency challenge, `tutor_id` and `time_range` are deliberately denormalized into `appointments`. This enables PostgreSQL to enforce an engine-level exclusion constraint (`EXCLUDE USING gist`) directly on the student's schedule.

- **Guaranteed Consistency via Composite Foreign Key**: Denormalization typically introduces the risk of data divergence (e.g., an appointment row referencing a slot ID but containing mismatched tutor or timing data). To eliminate this risk, `availability_slots` defines a composite unique constraint:

```
CONSTRAINT uq_slot_tutor_range UNIQUE (id, tutor_id, time_range);
```

`appointments` enforces a matching composite foreign key constraint:

```
FOREIGN KEY (slot_id, tutor_id, time_range)
    REFERENCES availability_slots (id, tutor_id, time_range)
    ON DELETE RESTRICT;
```

This guarantees that the denormalized columns can never diverge from their source slot, ensuring robust concurrency protection without compromising relational integrity.

4.2.3 Concurrency Control and Double-Booking Mitigation (NFR-03, BR-CONF-05)

A core requirement of Phase 2 is preventing race conditions during appointment scheduling. Concurrency protection is achieved through three complementary database-level mechanisms:

1. **Tutor Slot Overlap Prevention (`no_tutor_slot_overlap`)**: Enforced on `availability_slots` using PostgreSQL's native range types (`TSTZRANGE`) and the `btree_gist` extension:

```
CONSTRAINT no_tutor_slot_overlap EXCLUDE USING gist (
    tutor_id WITH =,
    time_range WITH &&
);
```

This ensures that no single tutor can publish overlapping availability intervals.

2. **Student Appointment Overlap Prevention (`no_student_appointment_overlap`)**: Enforced directly on `appointments`:

```
CONSTRAINT no_student_appointment_overlap EXCLUDE USING gist (
    student_id WITH =,
    time_range WITH &&
) WHERE (status <> 'CANCELLED');
```

This prevents a student from holding multiple active bookings that overlap in time, while allowing subsequent bookings if a previous reservation was cancelled.

3. **Active Slot Single-Booking Index (`uq_appointments_active_slot`)**: While the GiST exclusion constraint on tutor slots prevents slots from overlapping with one another, it does not prevent multiple appointment records from referencing the exact same slot ID. To guarantee that an active slot cannot be booked by more than one student concurrently (NFR-03), a partial unique index is enforced:

```
CREATE UNIQUE INDEX uq_appointments_active_slot
    ON appointments (slot_id) WHERE status <> 'CANCELLED';
```

Combined with row-level locking (`SELECT ... FOR UPDATE`) in booking transactions, these constraints provide deterministic, engine-level protection against double bookings and scheduling conflicts.

4.2.4 Data Dictionary and Schema Specifications

The schema uses specialized PostgreSQL types: `UUID` (anti-enumeration and IDOR mitigation), `TIMESTAMPTZ` (UTC standard), `TSTZRANGE` for time slots, and native `ENUM` types (`user_role`, `user_status`, `slot_status`, `appointment_status`, `session_type`) for every closed set of values, so an invalid state cannot be stored and the allowed values are declared once.

Table: `users`

Represents all system actors (Students, Tutors, and Administrators).

Column	Type	Nullable	Default	Description / Constraints
`id`	UUID	No	`gen_random_uuid()`	Primary Key
`email`	VARCHAR(255)	No	-	Unique, normalized lowercase
`password_hash`	VARCHAR(255)	No	-	Cryptographic hash (bcrypt, cost 12)
`first_name`	VARCHAR(100)	No	-	User first name
`last_name`	VARCHAR(100)	No	-	User last name
`role`	ENUM `user_role`	No	`'STUDENT'`	`'STUDENT'`, `'TUTOR'`, `'ADMIN'`
`status`	ENUM `user_status`	No	`'ACTIVE'`	`'ACTIVE'`, `'SUSPENDED'` (CU-13)
`failed_login_count`	INT	No	`0`	Consecutive failed login attempts (CU-02)

Column	Type	Nullable	Default	Description / Constraints
`locked_until`	TIMESTAMPTZ	Yes	`NULL`	Expiration timestamp for account lockout
`late_cancellation_count`	INT	No	`0`	Policy infraction accumulator (BR-CONF-02)
`created_at`	TIMESTAMPTZ	No	`CURRENT_TIMESTAMP`	Account creation timestamp
`updated_at`	TIMESTAMPTZ	No	`CURRENT_TIMESTAMP`	Managed via trigger `trg_users_updated_at`

Table: `subjects`

Catalog of available academic courses.

Column	Type	Nullable	Default	Description / Constraints
`id`	UUID	No	`gen_random_uuid()`	Primary Key
`code`	VARCHAR(50)	No	-	Unique subject identifier (e.g., `MAT-101`)
`name`	VARCHAR(255)	No	-	Full title of the academic subject
`department`	VARCHAR(255)	Yes	`NULL`	Associated academic department
`created_at`	TIMESTAMPTZ	No	`CURRENT_TIMESTAMP`	Record creation timestamp
`updated_at`	TIMESTAMPTZ	No	`CURRENT_TIMESTAMP`	Managed via trigger `trg_subjects_updated_at`

Table: `tutor_profiles`

Profile information for users registered as tutors (1:1 with `users`).

Column	Type	Nullable	Default	Description / Constraints
`user_id`	UUID	No	-	Primary Key, FK -> `users(id)` ON DELETE CASCADE
`bio`	TEXT	Yes	`NULL`	Professional bio (at most 1000 characters, validated in `tutorService`)
`meeting_url`	VARCHAR(500)	Yes	`NULL`	HTTPS video meeting link (Google Meet, Zoom, Teams)
`created_at`	TIMESTAMPTZ	No	`CURRENT_TIMESTAMP`	Profile creation timestamp
`updated_at`	TIMESTAMPTZ	No	`CURRENT_TIMESTAMP`	Managed via trigger `trg_tutor_profiles_updated_at`

Table: `tutor_subjects`

Junction table mapping tutors to their qualified subjects (N:M relationship).

Column	Type	Nullable	Default	Description / Constraints
`tutor_id`	UUID	No	-	Composite PK, FK -> `users(id)` ON DELETE CASCADE
`subject_id`	UUID	No	-	Composite PK, FK -> `subjects(id)` ON DELETE CASCADE
`created_at`	TIMESTAMPTZ	No	`CURRENT_TIMESTAMP`	Qualification assignment timestamp

Table: `availability_slots`

Available one-hour timeslots published by tutors.

Column	Type	Nullable	Default	Description / Constraints
`id`	UUID	No	`gen_random_uuid()`	Primary Key
`tutor_id`	UUID	No	-	FK -> `users(id)` ON DELETE CASCADE
`time_range`	TSTZRANGE	No	-	Half-open `[start, end)` timestamp interval
`status`	ENUM `slot_status`	No	`'AVAILABLE'`	`'AVAILABLE'`, `'BOOKED'`, `'UNAVAILABLE'`
`created_at`	TIMESTAMPTZ	No	`CURRENT_TIMESTAMP`	Creation timestamp
`updated_at`	TIMESTAMPTZ	No	`CURRENT_TIMESTAMP`	Managed via trigger `trg_availability_slots_updated_at`

- **Key Constraints**:
  - `chk_slot_duration_60m`: `CHECK (upper(time_range) - lower(time_range) = INTERVAL '60 minutes')`
  - `no_tutor_slot_overlap`: `EXCLUDE USING gist (tutor_id WITH =, time_range WITH &&)`
  - `uq_slot_tutor_range`: `UNIQUE (id, tutor_id, time_range)`

Table: `appointments`

Scheduled tutoring sessions booked by students.

Column	Type	Nullable	Default	Description / Constraints
`id`	UUID	No	`gen_random_uuid()`	Primary Key
`slot_id`	UUID	No	-	Part of the composite FK to `availability_slots`

Column	Type	Nullable	Default	Description / Constraints
`tutor_id`	UUID	No	-	Part of the composite FK (Denormalized)
`student_id`	UUID	No	-	FK -> `users(id)` ON DELETE CASCADE
`time_range`	TSTZRANGE	No	-	Part of the composite FK (Denormalized)
`session_type`	ENUM `session_type`	No	`'STANDARD'`	`'QUICK_CHECK'` (5 min), `'STANDARD'` (30 min), `'DEEP_DIVE'` (60 min); always one 60-minute block (BR-CONF-03)
`status`	ENUM `appointment_status`	No	`'CONFIRMED'`	`'CONFIRMED'`, `'COMPLETED'`, `'CANCELLED'`
`student_inquiry`	TEXT	Yes	`NULL`	Student consultation notes/questions
`tutor_notes`	TEXT	Yes	`NULL`	Private notes recorded by tutor after session
`student_rating`	SMALLINT	Yes	`NULL`	Rating from 1 to 5 (`CHECK`); `NULL` until the student rates the session (CU-12)
`student_feedback`	TEXT	Yes	`NULL`	Optional written feedback (CU-12)
`reminder_sent`	BOOLEAN	No	`FALSE`	Set when the email provider accepts the reminder (CU-10)
`cancellation_reason`	VARCHAR(500)	Yes	`NULL`	Optional reason given when cancelling (CU-09)
`is_late_cancellation`	BOOLEAN	No	`FALSE`	Set when cancelled less than 4 hours before the start (BR-CONF-02)
`cancelled_by`	UUID	Yes	`NULL`	FK -> `users(id)` tracking cancellation initiator
`cancelled_at`	TIMESTAMPTZ	Yes	`NULL`	Cancellation timestamp
`created_at`	TIMESTAMPTZ	No	`CURRENT_TIMESTAMP`	Booking timestamp
`updated_at`	TIMESTAMPTZ	No	`CURRENT_TIMESTAMP`	Managed via trigger `trg_appointments_updated_at`

- **Key Constraints**:

- `FOREIGN KEY (slot_id, tutor_id, time_range) REFERENCES availability_slots(id, tutor_id, time_range) ON DELETE RESTRICT`
- `no_student_appointment_overlap`: `EXCLUDE USING gist (student_id WITH =, time_range WITH &&) WHERE (status <> 'CANCELLED')`
- `uq_appointments_active_slot`: `UNIQUE INDEX (slot_id) WHERE status <> 'CANCELLED'`

Table: `audit_logs`

Append-only log documenting security-relevant events and status transitions (CU-14).

Column	Type	Nullable	Default	Description / Constraints
`id`	UUID	No	`gen_random_uuid()`	Primary Key
`actor_id`	UUID	Yes	`NULL`	FK -> `users(id)` ON DELETE SET NULL
`action`	VARCHAR(100)	No	-	Action identifier (e.g., `USER_SUSPENDED`)
`target_type`	VARCHAR(100)	No	-	Affected entity category (`USER`, `APPOINTMENT`)
`target_id`	UUID	Yes	`NULL`	Identifier of the affected entity
`details`	JSONB	Yes	`NULL`	Structured contextual metadata
`created_at`	TIMESTAMPTZ	No	`CURRENT_TIMESTAMP`	Event timestamp

- **Key Constraints**:
  - `trg_audit_logs_immutable`: Trigger executing `prevent_audit_logs_mutation()`, throwing an exception on any attempted `UPDATE` or `DELETE`.

4.2.5 Automated Database Migrations & Seed Architecture

Migration Engine (`src/database/migrate.js`)

Database schema migrations are managed through version-controlled, sequential SQL files located in `src/database/migrations/`:

- **State Tracking**: The table `schema_migrations (id SERIAL PRIMARY KEY, filename VARCHAR(255) NOT NULL UNIQUE, applied_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP)` records every executed script.
- **Transactional Atomicity**: Each migration file is read and executed inside an explicit `BEGIN ... COMMIT` block. If any statement fails, the transaction is rolled back immediately via `ROLLBACK`, leaving the schema in a clean, consistent state.
- **Idempotency**: Running `npm run db:migrate` repeatedly is safe; migrations already present in `schema_migrations` are automatically skipped.

- **Current migrations**: `001_initial_schema.sql` (types, tables, constraints and triggers), `002_add_indexes.sql` (secondary indexes, Section 4.2.6), `003_add_session_type.sql` (the `session_type` ENUM and column), `004_add_unaccent.sql` (the `unaccent` extension, so the CU-06 search ignores accents: `unaccent(u.first_name) ILIKE unaccent($1)` lets "garcia" find "García" and "algebra" find "Álgebra Lineal") and `005_add_reminder_rating_cancel_reason.sql` (the columns that CU-09, CU-10 and CU-12 need). The last three show the rule the team follows: a migration that is already in `main` is never edited, because databases that applied it would not run it again; every change is a new numbered file.

Seed Strategy (`src/database/seeds.sql`)

Test and development datasets are loaded via `npm run db:seed`:

- **Protection**: The seed empties every table and its password is public, so the script refuses to run in production or against a remote database unless `ALLOW_SEED_RESET=true` is set.
- **Cryptographic Hashes**: Seed accounts use genuine bcrypt password hashes (cost factor 10) matching the standard testing credential `Password123!`.
- **Standardized Roles**: Creates an administrator (`admin@unimentor.edu`), certified tutors (`laura.garcia@unimentor.edu`, `carlos.martin@unimentor.edu`), and students.
- **Realistic Scheduling Scenarios**: Slots are placed at round hours with at least two hours of notice. Appointments are populated covering all three valid lifecycle states: one active confirmed future session, one completed past session with notes and feedback, and one cancelled appointment tracking `cancelled_by` and `cancelled_at`.

4.2.6 Query Optimization and Indexing Strategy

Many lookups are already served by indexes that PostgreSQL creates for the constraints: the primary keys, `UNIQUE (email)`, `UNIQUE (code)`, the GiST indexes behind the two `EXCLUDE` constraints and the partial unique index `uq_appointments_active_slot`. Migrations `002_add_indexes.sql` and `005_add_reminder_rating_cancel_reason.sql` add only the indexes that the main read paths still need to stay under 200 ms (NFR-01):

Index	Columns	Used by
`idx_tutor_subjects_subject`	`tutor_subjects (subject_id)`	CU-06: tutors of a subject (the primary key starts with `tutor_id`, so it cannot serve this lookup)
`idx_appointments_student`	`appointments (student_id)`	CU-11: the student's dashboard
`idx_appointments_tutor`	`appointments (tutor_id)`	CU-11: the tutor's dashboard
`idx_availability_slots_available`	`availability_slots (tutor_id, lower(time_range)) WHERE status = 'AVAILABLE'`	CU-06 and CU-07: the next free slot of a tutor. Being partial, it only stores free slots and stays small

Index	Columns	Used by
`idx_appointments_pending_reminder`	`appointments (lower(time_range)) WHERE status = 'CONFIRMED' AND NOT reminder_sent`	CU-10: the reminders still pending, every 10 minutes

No index was added for columns that are not filtered on, because every index slows down writes and the booking transaction (CU-08) is the most write-sensitive path of the system.

4.3 Design Patterns

This section documents the design patterns applied in the implementation. Each one answers a concrete problem of the code base: **Chain of Responsibility** for the request security pipeline (4.3.1, authentication and administration), **Repository** for data access (4.3.2, bookings) and **Adapter/Strategy** for notifications (4.3.3, `NotificationService`, ADR 6).

4.3.1 Chain of Responsibility: the Request Security Pipeline

Type: behavioral (GoF). **Use cases:** CU-01, CU-02, CU-13, CU-14, and every protected route of the other modules (e.g. CU-03).

Problem

Before a protected request reaches its controller, several independent checks must pass, always in the same order: the request must not be a cross-site forgery (ADR 4), it must carry a valid session, the account must still be active (CU-13: "*Suspended accounts are rejected on every authenticated request*") and the user must have the right role. For example, `POST /api/admin/users/:id/suspend` needs all four. Writing these checks inside each controller would duplicate them in every endpoint of the five modules, mix security with business logic, break the Single Responsibility Principle, and make it easy to forget one check in one route: a security hole that no test of the other routes would detect.

Solution

Each check is an independent **handler** with the same signature, `(req, res, next)`. A handler either passes the request to the next one by calling `next()`, or stops the chain by throwing an `AppError`. Express is the mechanism that links them, so no extra framework is needed:

GoF role	UniMentor element
Handler (common interface)	The Express middleware signature `(req, res, next)`
ConcreteHandler	`csrfProtection` (`csrfMiddleware.js`), `rejectNullBytes` (`inputMiddleware.js`), `verifyJWT` and `requireRole(...roles)` (`authMiddleware.js`), and the final controller

GoF role	UniMentor element
Successor link	Calling `next()`
Stopping the chain	`throw new AppError(status, message, code)`. Express 5 sends it, also from `async` handlers, straight to `errorHandler.js`, the last link (Section 4.1.4), which writes the JSON error
Client (builds the chain)	`app.js` and the routers (`adminRoutes.js`, `tutorRoutes.js`...)

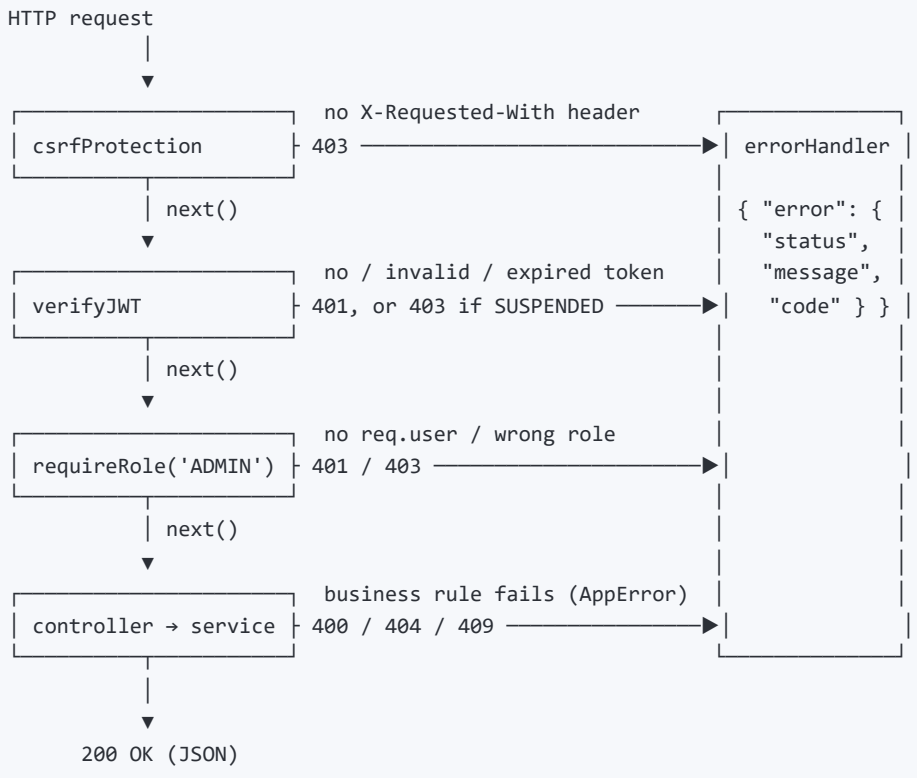
The chain is assembled in two places of the real code. `csrfProtection` and `rejectNullBytes` are mounted once for the whole API, so no route can forget them, and each router adds the links it needs:

```
// app.js: first links for every API route
app.use('/api', csrfProtection, rejectNullBytes, apiRoutes);

// adminRoutes.js: the whole router requires an active ADMIN account (CU-13, CU-14)
router.use(verifyJWT, requireRole('ADMIN'));
router.post('/users/:id/suspend', adminController.suspendUser);

// tutorRoutes.js: only the signed-in tutor edits their profile (CU-03)
router.put('/profile', verifyJWT, requireRole('TUTOR'), tutorController.updateProfile);
```

`requireRole` is a handler *factory*: `requireRole('ADMIN')` and `requireRole('TUTOR')` are two different links built from the same code, so P4 and P5 protect their routes with `requireRole('STUDENT')` or `requireRole('TUTOR')` without writing new checks.



What each link does

- **`csrfProtection` (ADR 4, NFR-02).** The session cookie is `SameSite=None` because the frontend (Vercel) and the API (Render) are on different sites, so a forged request from another site would also carry it. The link lets `GET`, `HEAD` and `OPTIONS` through and requires the `X-Requested-With: XMLHttpRequest` header on every other method. A cross-site page cannot add a custom header without a CORS preflight, and CORS only accepts the configured frontend origins. `frontend/js/api.js` adds the header to every request. It also protects `/auth/login` against *login CSRF*.
- **`rejectNullBytes`.** Answers 400 if the query or the body contains a NUL character, which PostgreSQL cannot store in text and would otherwise end as a 500. It is omitted from the diagram above, between `csrfProtection` and `verifyJWT`.
- **`verifyJWT`.** Reads the token from the `HttpOnly` cookie (never from `localStorage` or the `Authorization` header, which ADR 4 rejected), verifies the signature and expiry with `HS256` (clearing the cookie if they fail, CU-11 A1), and then reads `id, role, status` from `users`. A `SUSPENDED` account is rejected with 403 even if its token is still valid, which is how CU-13 "*invalidates any JWT already issued*". The role also comes from the database, so a role change applies immediately. It leaves `req.user = { id, role }` for the next links.
- **`requireRole(...roles)`.** Answers 401 if no previous link authenticated the request and 403 if the role is not allowed (CU-13, A1). The roles are the values of the `user_role` ENUM.

Why the order matters

The order is part of the design. `csrfProtection` comes first because it is the cheapest check and rejects forged requests before any database query. `verifyJWT` must come before `requireRole`, because it is the link that creates `req.user`; in the opposite order every request would get a 401.

Why the login lockout is not a link

The CU-02 rule (*5 consecutive failed attempts lock the account for 15 minutes*) looks like a candidate for another middleware, such as an IP rate limiter, but it cannot be one. It must count only *failed* attempts, so it needs to know whether `bcrypt.compare` succeeded, which happens inside `authService`. It must also count per account and not per IP, because many students share the university network, and survive restarts of the Render service. It is therefore a business rule in the service, stored in `users.failed_login_count` and `users.locked_until` and updated with a single atomic `UPDATE`.

Consequences

Benefits

- **Separation of concerns.** Controllers and services only contain business logic; they can assume that `req.user` exists and has the right role.

- **Reuse.** Each link is written once and combined per route. The P2, P4 and P5 routes are protected with one line.
- **Secure by default.** The CSRF link is mounted globally, and the admin router applies `verifyJWT` and `requireRole('ADMIN')` to all its routes, so a new endpoint is protected without extra code.
- **Testability.** Each link is a plain function that is tested on its own (`tests/unit/authMiddleware.test.js`), and the whole chain is tested over HTTP with Supertest (`tests/integration/adminRoutes.test.js`): no cookie gives 401, a student gives 403, and a POST without the header gives 403 before the database is queried.
- **One error format.** Every rejection reaches the same `errorHandler`, so the frontend handles 401, 403 and 429 in one place (`api.js`).

Drawbacks

- **The order depends on whoever assembles the route.** Swapping two links breaks the security silently. It is mitigated by mounting the common links at router or application level and by the integration tests.
- **A link that neither calls `next()` nor throws leaves the request hanging** until the client times out.
- **Harder debugging.** The flow of a request is spread across several files instead of one function.
- **One database query per authenticated request.** `verifyJWT` reads the account status on every call. It is a primary-key lookup, but it is the price of being able to revoke the sessions of suspended accounts at once (CU-13), which a purely stateless JWT could not do.

Related decisions

ADR 4 (JWT in an `HttpOnly` cookie, cross-site CORS with credentials and CSRF protection), NFR-02 (bcrypt with cost 12, signed JWT, cookie flags `HttpOnly; Secure; SameSite=None` in production, checked by the integration tests), CU-13 (account status checked on every request) and CU-14 (the critical actions are written to the append-only `audit_logs` table inside the same transaction as the action, so a suspension can never exist without its audit entry).

4.3.2 Repository: Isolating Data Access

Type: architectural (Fowler, *Patterns of Enterprise Application Architecture*). **Use cases:** every use case that reads or writes data; the booking module (CU-04, CU-05, CU-07, CU-08, CU-09) is where it matters most.

Status: applied in all the code merged so far (`userRepository`, `tutorRepository`, `subjectRepository`, `adminRepository`, `auditLogRepository`, `healthRepository`). The booking repository is the next one to be written, and its code below is the planned design for CU-08.

Problem

The booking flow mixes two kinds of knowledge: business rules (BR-CONF-01: two hours of advance notice; BR-CONF-04: at most three active bookings) and persistence details (`SELECT ... FOR UPDATE`, `TSTZRANGE`, the composite foreign key of Section 4.2). If services contained SQL, a schema change would

spread through the business logic, the same query would be repeated in several use cases, and no rule could be unit-tested without a running database. Concatenating SQL in many places would also make it harder to check that every query is parameterized (NFR-02).

Solution

Each aggregate gets one module in `repositories/` that exposes intention-revealing functions and hides the SQL behind them. It is the only layer that writes SQL, and it returns plain objects. Because JavaScript has no interfaces, the contract is the set of exported functions, which is exactly what the tests replace with `jest.mock`.

Role	UniMentor element
Repository	`repositories/*Repository.js` (real), `appointmentRepository.js` (planned)
Client	The services: `authService`, `adminService`, `bookingService` (planned)
Unit of work	`db.withTransaction(work)`, which gives `work` a dedicated client between `BEGIN` and `COMMIT` or `ROLLBACK`

The important design decision is how a repository takes part in a transaction. A use case such as CU-13 spans several repositories, so the service opens the transaction and passes the client to each function. This is the real suspension flow, shortened:

```
// adminService.js: the service decides WHAT happens and in which order
return db.withTransaction(async (client) => {
  const target = await userRepository.lockById(client, targetId); // SELECT ... FOR UPDATE
  const user = await userRepository.updateStatus(client, targetId, 'SUSPENDED');
  const cancelled = await adminRepository.cancelFutureAppointments(client, targetId, adminId);
  await adminRepository.releaseSlots(client, cancelled.map((a) => a.slot_id), targetId);
  await auditLogRepository.insert({ actorId: adminId, action: 'USER_SUSPENDED' /* ... */ }, client);
  return { user, cancelledAppointments: cancelled };
});
```

The booking transaction of CU-08 (planned) uses the same shape and follows the sequence diagram of Section 3.2.2-A. Locking the student row serializes parallel bookings by one student, so the quota cannot be bypassed; locking the slot row makes the second of two simultaneous requests see `BOOKED` and answer 409:

```
// bookingService.js (planned): rules here, SQL in the repositories
return db.withTransaction(async (client) => {
  await userRepository.lockById(client, studentId);
  const slot = await appointmentRepository.lockSlot(client, slotId); // SELECT ... FOR UPDATE
  if (slot.status !== 'AVAILABLE') throw new AppError(409, SLOT_TAKEN);          // CU-08, A1
  if (startsInLessThanTwoHours(slot)) throw new AppError(400, NOTICE_TOO_SHORT); // BR-CONF-01
  if ((await appointmentRepository.countActive(client, studentId)) >= 3) {       // BR-CONF-04
    throw new AppError(400, QUOTA_EXCEEDED);
  }
  await appointmentRepository.markSlotBooked(client, slotId);
```

```
  return appointmentRepository.create(client, { slot, studentId, sessionType, inquiry });
});
```

Consequences

Benefits

- **Separation of concerns.** Rules live in services and SQL lives in repositories, so a schema change such as migration `003_add_session_type.sql` only touches the repositories that use the column.
- **Testability.** Services are unit-tested with mocked repositories, without PostgreSQL (`tests/unit/adminService.test.js`). Real SQL needs its own tests against a real database, because a mock cannot detect a wrong query.
- **One place to audit.** Parameterized queries (NFR-02) and locks (NFR-03) are reviewed in a single folder.
- **Reuse.** `lockById` serves both suspending an account and, as planned, serializing bookings.

Drawbacks

- **More files and indirection.** A simple read still crosses four layers.
- **Business rules can leak.** A `WHERE status = 'CONFIRMED'` is a persistence filter that also encodes a rule. We accept this where the database must do the filtering.
- **The abstraction is not total.** Services must know that a transaction exists and pass `client` along, which is the price of letting one transaction span several repositories.

Why not an ORM. Sequelize or Prisma would generate the SQL, but UniMentor depends on PostgreSQL features that are clearer in plain SQL: `TSTZRANGE`, `EXCLUDE` constraints and explicit row locks (ADR 2). Hand-written repositories keep those visible.

Related decisions

ADR 2 (PostgreSQL and `SELECT ... FOR UPDATE`), NFR-02 (parameterized queries), NFR-03 (concurrent bookings), NFR-04 (layered architecture) and BR-CONF-05 (Section 4.2.3: the database constraints remain the last line of defence if a rule in the service were ever wrong).

4.3.3 Adapter and Strategy: the Notification Service

Type: structural (Adapter) and behavioral (Strategy), both GoF. **Use cases:** CU-01 (welcome email), CU-08 (confirmation), CU-09 and CU-13 (cancellation notices) and CU-10 (reminders).

Status: design. `NotificationService` is not implemented yet; the code below shows the structure that will be built to meet ADR 6. The points that will call it are already marked in `authService.js` and `adminService.js`.

Problem

ADR 6 chose SendGrid but requires the provider to stay behind a single interface. Calling the SendGrid SDK directly from the services would cause three problems. Business logic would depend on a vendor's API, so changing provider would touch every use case. Tests and local development would send real email or need an API key. And a provider failure could undo a booking that is already valid, while the requirements treat email as asynchronous: the booking commits first (CU-08, steps 7 and 8), and a failed reminder is simply retried in the next cycle (CU-10, A1).

Solution

Two patterns solve different parts of the problem. **Adapter** converts the interface we want into the one SendGrid offers. **Strategy** makes the delivery channel interchangeable: the service holds one provider and does not know which one it is, and the provider is chosen once, from the configuration, when the application starts.

GoF role	UniMentor element
Target / Strategy interface	An email provider: an object with `send({ to, subject, text, html })`
Adapter / ConcreteStrategy	`SendGridProvider` (wraps `@sendgrid/mail`)
ConcreteStrategy	`ConsoleProvider` (logs the message, for development and tests)
Adaptee	The SendGrid SDK
Context / Client	`NotificationService`, used by the services and by the CU-10 scheduler

```
// SendGridProvider.js: the only module that knows the SDK
class SendGridProvider {
  constructor(client, from) { this.client = client; this.from = from; }
  async send({ to, subject, text, html }) {
    await this.client.send({ to, from: this.from, subject, text, html });
  }
}

// NotificationService.js: the use cases speak its language, not the vendor's
class NotificationService {
  constructor(provider) { this.provider = provider; } // injected Strategy
  async sendBookingConfirmation(appointment) {
    const message = bookingTemplate(appointment);
    await Promise.all([
      this.provider.send({ to: appointment.studentEmail, ...message }),
      this.provider.send({ to: appointment.tutorEmail, ...message }),
    ]);
  }
}
```

Failures are handled according to each use case, using the same provider. After a `COMMIT`, a failed confirmation is logged and does not affect the response, because the booking is already valid. In CU-10 the scheduler sets `reminder_sent = TRUE` only after the provider accepts the message (the sequence diagram of Section 3.2.2), so a failure leaves the appointment queued for the next 10-minute cycle.

Consequences

Benefits

- **Vendor independence.** Replacing SendGrid means writing one new provider class. No use case changes, which is the impact ADR 6 asks for.
- **Testability.** Tests inject a fake provider and check which messages would be sent, with no network and no API key.
- **Safe development.** Without an API key, `ConsoleProvider` is selected and no real email can leave a developer's machine.
- **Open/Closed.** A new channel, such as SMS, is a new class that implements the same interface.

Drawbacks

- **More classes** for what currently is a single provider. We accept them because ADR 6 explicitly requires the boundary, and the fake provider pays for itself in the tests.
- **The interface is a lowest common denominator.** Provider-specific features such as SendGrid's dynamic templates are not exposed; our templates are rendered in our own code.
- **Asynchronous failures are less visible.** An email lost after the `COMMIT` does not reach the user as an error, so failures must be logged.

Related decisions

ADR 6 (SendGrid behind a single interface), goal G5 (confirmation within 60 seconds, reminders 24 hours before with a tolerance of 15 minutes), CU-10 (retries through `reminder_sent`) and NFR-04 (maintainability).